RESEARCH-ORIENTED UNIVERSITY SYSTEM

Discipline: Object-Oriented Programming and Design

Project type: Console-based Java Application

University: KBTU

Team members:

Abdrakhman Diana

Yerdauit Yerlanuly

Margulan Zhaskairatuly

Yeraly Zharylkassyn

MAIN GOAL OF THE PROJECT

The main goal oF this project is to create a console-based inFormation system For a research-oriented university

The system helps manage academic, administrative, and research processes inside the university

It allows diFFerent users to log in and perForm actions according to their roles

Main system areas:

- *User authentication*
- *Course registration*
- *Teacher and student management*
- *Marks and transcript generation*
- *Research papers and research projects*
- *Reports, messages, news, and logs*
- *Data storage using serialization*

PROJECT DESCRIPTION

The project represents a university intranet system. It is implemented as a console-based Java application

The system includes several types of users:

- Student
- Teacher
- Manager
- Admin
- Employee
- Researcher

Each role has its own responsibilities

For example, students can register for courses, teachers can put marks, managers can approve registration, admins can manage users, and researchers can manage papers and projects

```
User logs in → Chooses action → System checks role → Executes operation
```

USE CASE DIAGRAM

The Use Case diagram shows how actors interact with the system

Main actors:

- User
- Student
- Employee
- Teacher
- Manager
- Admin
- Researcher

Main use cases:

- Login / Log out
- View profile
- Register for courses
- Approve student registration
- Assign courses to teachers
- Put marks
- View transcript
- Add research paper
- Join research project
- Manage users
- View action logs

UML CLASS DIAGRAM

The UML Class diagram describes the main classes, inheritance, interfaces, and relationships

Key design decisions:

- User is the base abstract class.
- Student and Employee inherit from User.
- Teacher, Manager, and Admin inherit from Employee.
- Researcher is an interface.
- Professor, ResearchStudent, and ResearchEmployee implement Researcher.
- Course can have more than one instructor.
- ResearchProject accepts only researchers.

PACKAGE STRUCTURE

Main responsibilities:

- ***users*** -> user hierarchy and roles;
- ***academic*** -> courses, lessons, marks, transcripts;
- ***research*** -> papers, projects, research proFiles;
- ***services*** -> business logic;
- ***storage*** -> data storage and serialization;
- ***exceptions*** -> custom exceptions

```
src/
  ├── users/
  ├── academic/
  ├── research/
  ├── services/
  ├── storage/
  ├── exceptions/
  ├── enums/
  ├── comparators/
  ├── communication/
  ├── reports/
  ├── logs/
  ├── factories/
  ├── facade/
  └── Main.java
```

USER HIERARCHY

The system uses inheritance to represent different types of users

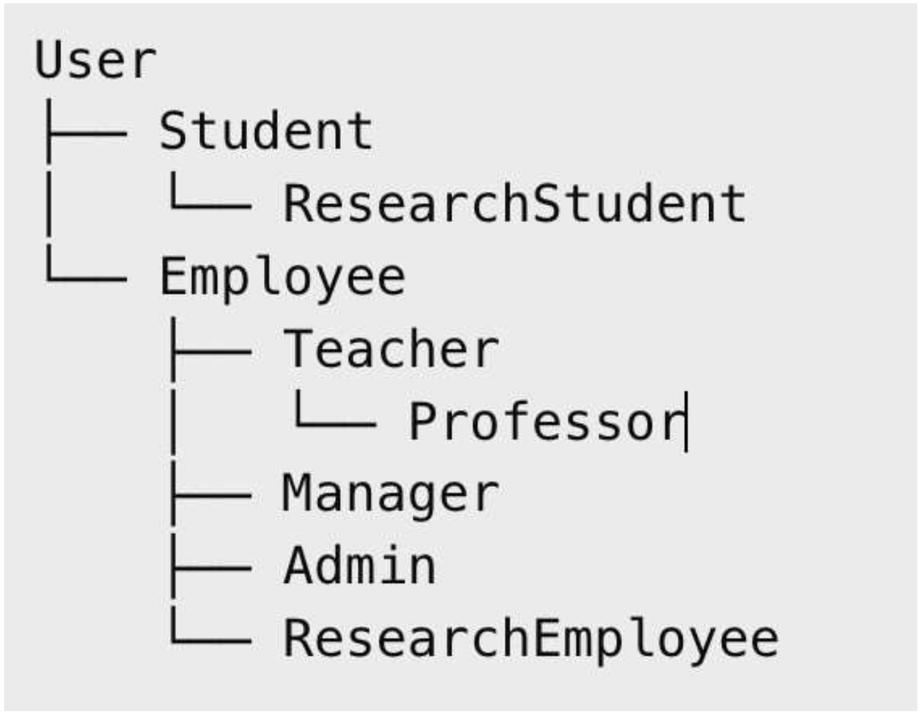
User stores common fields: id, username, password, name, email, and role

Student represents a bachelor student

Employee is a base class for university workers

Teacher, Manager, and Admin are specialized employees

Professor is a teacher who is always a researcher



ACADEMIC MODULE

The academic module manages courses, lessons, registration, marks, and transcripts

Main classes:

- Course
- Lesson
- Mark
- Transcript
- RegistrationRequest

Important rules:

- A course can have more than one instructor
- Lesson type can be LECTURE or PRACTICE
- A student cannot take more than 21 credits
- Marks consist of first attestation, second attestation, and final exam

```
Student sends request → Manager approves → Student is added to course
```

RESEARCH MODULE

The research module is the key part of the research-oriented university system

Main classes:

- Researcher
- ResearchProfile
- ResearchPaper
- ResearchProject

Why Researcher is an interface?

Researcher is a role, not a fixed user type. A student, teacher, or employee can be a researcher. Java does not support multiple inheritance of classes, so an interface is the best solution

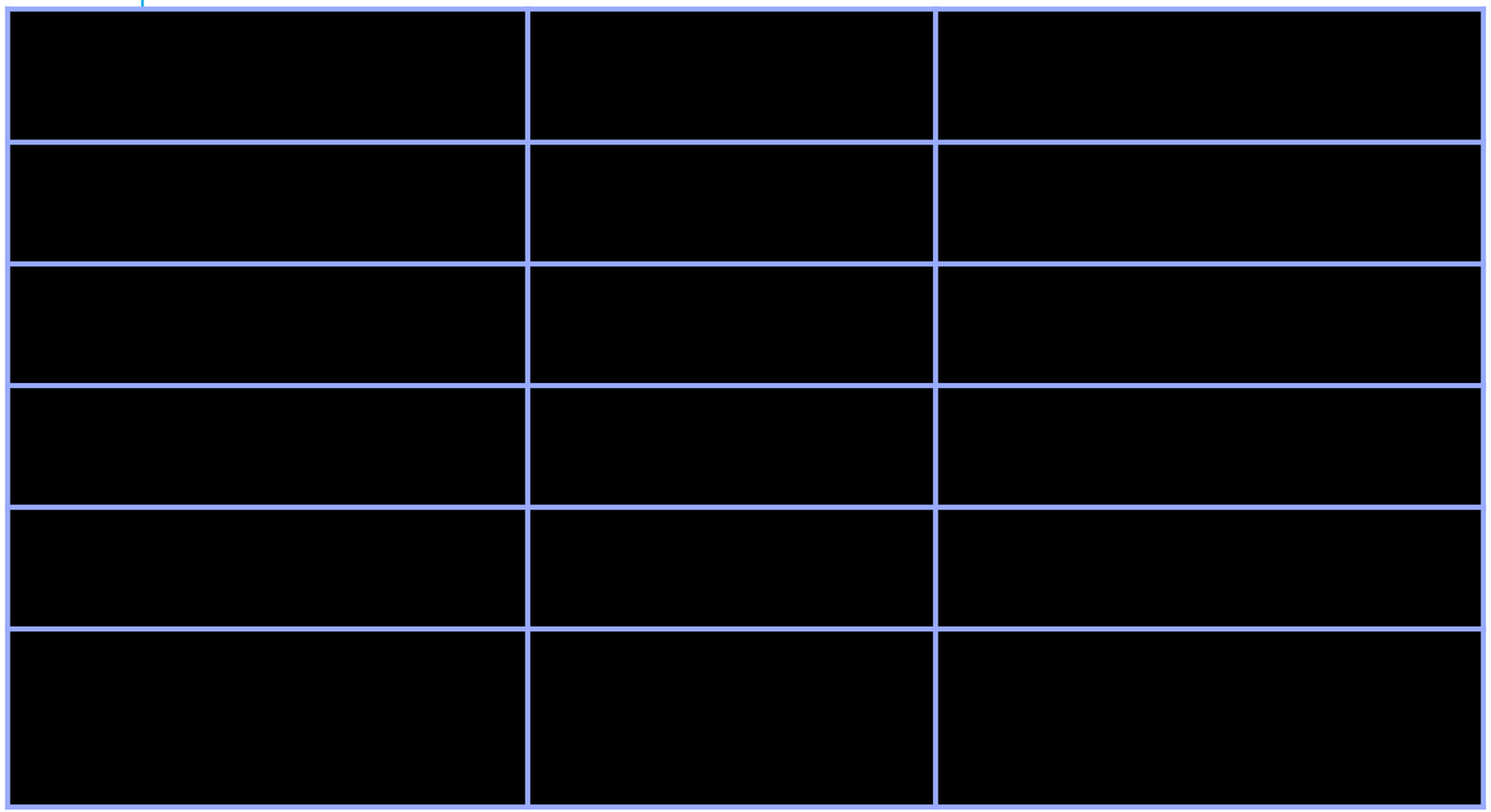
Important rules:

- Professor is always a researcher
- Supervisor must have h-index at least 3
- Only researchers can join research projects

```
public interface Researcher {
    ResearchProfile getResearchProfile();
    void printPapers(Comparator<ResearchPaper> comparator);
}
```

DESIGN PATTERNS

The project uses five design patterns



Research papers can be sorted using different comparator strategies:

```
new PaperCitationsComparator();
new PaperDateComparator();
new PaperPagesComparator();
```

EXCEPTIONS, SERIALIZATION, AND AUTHENTICATION

The system uses custom exceptions for invalid operations:

- AuthenticationException
- CreditLimitExceededException
- LowHIndexException
- NotResearcherException
- MarkValidationException
- RegistrationRejectedException

The system uses Java serialization to save and load data. **Main model classes implement Serializable**

All users must log in through AuthService

DEMO, DOCUMENTATION, AND TESTING

The Main class demonstrates the main workflow:

1. Creating users
2. Logging in
3. Creating courses
4. Assigning teachers
5. Student registration request
6. Manager approval
7. Teacher putting marks
8. Transcript generation
9. Research paper sorting
10. Research project participation
11. Custom exception demonstration
12. Saving/loading data

HTML documentation was generated using JavaDoc

```
docs/index.html
```

The documentation includes class descriptions, constructors, fields, and methods

PROBLEMS

Problems Faced:

Researcher design

1. The main challenge was deciding how to model Researcher. The final solution was to make it an interface and store research data in ResearchProfile

UML and code consistency

2. The code had to follow the Use Case and UML diagrams. Some classes were added to improve the architecture: Professor, ResearchStudent, ResearchEmployee, and ResearchProfile

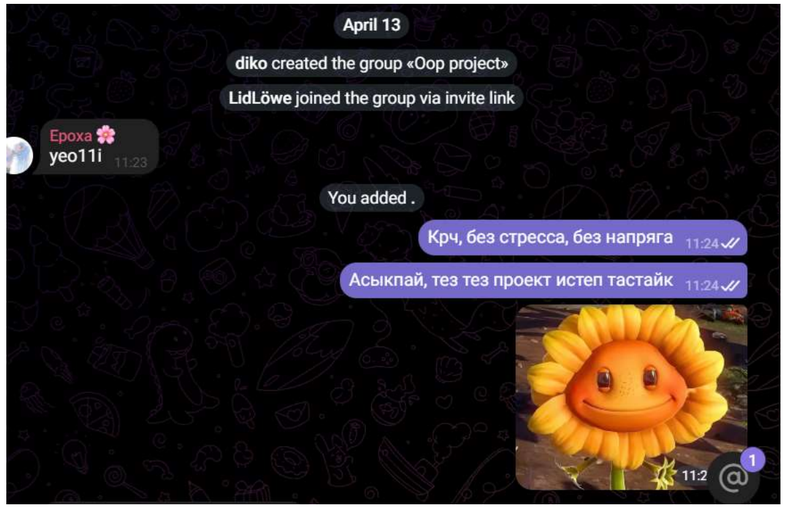
Serialization

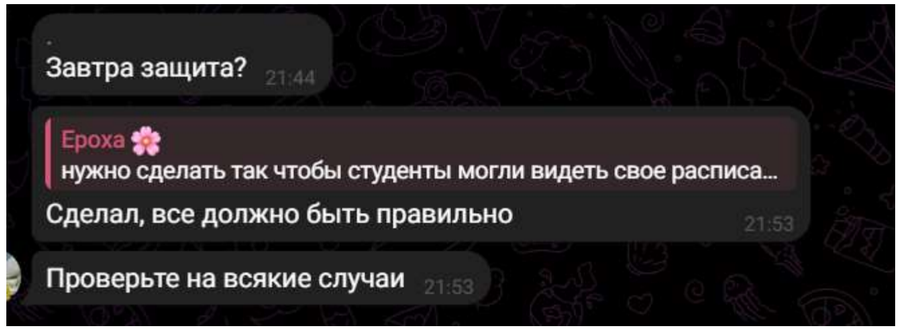
3. All connected model classes needed to implement Serializable to make saving and loading work correctly

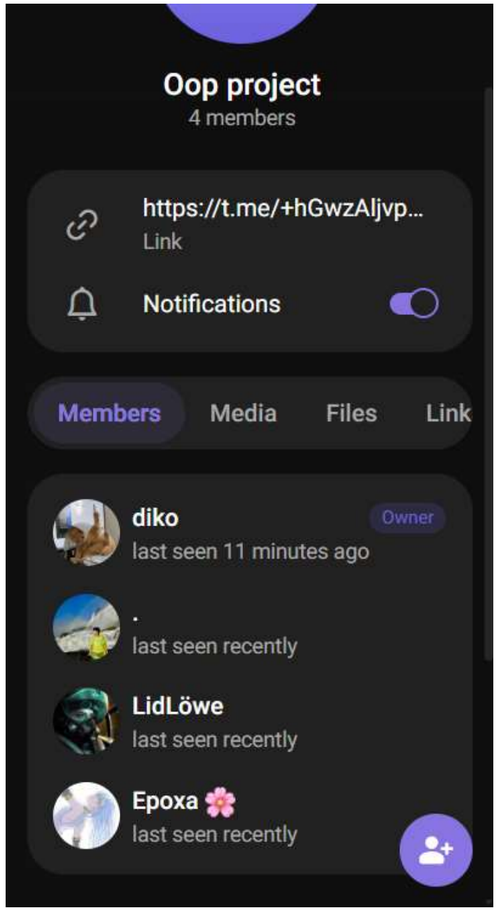
PROJECT MANAGEMENT

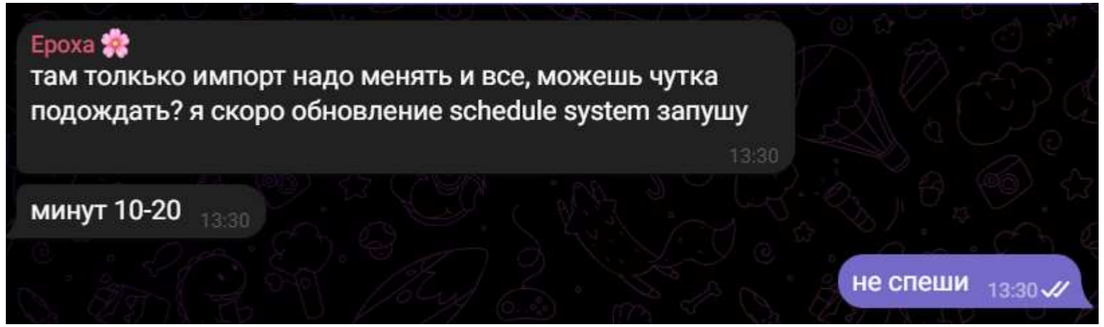
The work was completed step by step:

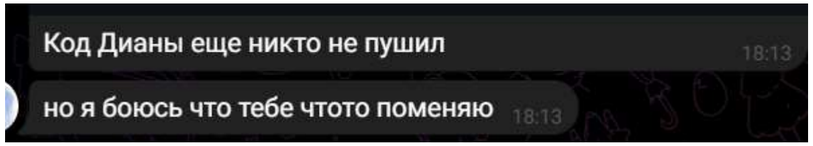
Requirements -> Diagrams -> Draft classes -> Full code -> Demo -> Documentation -> Report

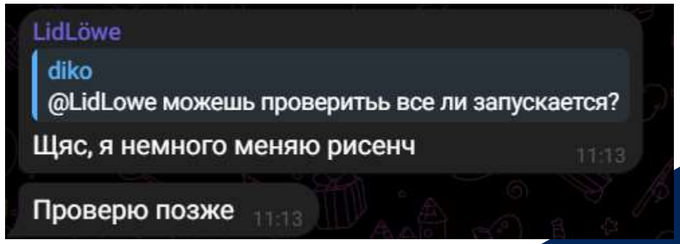












CONCLUSION

THE PROJECT SUCCESSFULLY IMPLEMENTS THE MAIN REQUIREMENTS OF THE OOP AND DESIGN FINAL PROJECT

It demonstrates ***inheritance, interfaces, polymorphism, encapsulation, custom exceptions, collections, comparators, serialization, and design patterns***

The final system is consistent with the UML and Use Case diagrams and can be demonstrated through the console

THANK YOU FOR YOUR ATTENTION!